

Interview cheat sheet: all lessons

Every lesson's cheat sheet in one place. ♦ = my addition beyond the video; everything else is from the lessons.

01 · What Is Inference & Inference Engineering?

[Full notes](#)

Say it in 30 seconds

Inference is running a trained model token by token for every request, so fleet-wide it costs more than training. Decode is memory-bandwidth-bound: every step streams all the weights to make one token per sequence. The core trade is latency vs throughput vs cost, and batching is lever one. The job: measure, find the bottleneck, change one thing, repeat.

Only numbers worth memorizing

- **H100**: 3.35 TB/s HBM, ♦ ~989 TFLOPS BF16. **Llama 3 8B**: 16 GB in BF16, ~128 KB KV per token (taught in 02).
- **Software matters**: batching gives ~15× cheaper tokens; vLLM gets up to 24× naive HF throughput.

Derive, don't memorize

Let P = parameters, B = batch size, BW = memory bandwidth.

1. Decode speed = bandwidth ÷ bytes per step

Every decode step streams all the weights once, so time per step is bytes moved over bytes per second.

$$t_{\text{step}} \approx \frac{2P \text{ bytes}}{BW} = \frac{16 \text{ GB}}{3.35 \text{ TB/s}} \approx 4.8 \text{ ms} \Rightarrow \leq 200 \text{ tok/s per user}$$

INTUITION

Halve the bytes (8-bit weights) and the ceiling doubles. That's why precision is a speed lever.

2. Memory-bound or compute-bound? Compare FLOPs per byte to the ridge ♦

Each weight does a multiply and an add (2 FLOPs) per token and costs 2 bytes to read. One read is shared by all B sequences:

$$\frac{\text{FLOPs}}{\text{byte}} \approx \frac{2P \cdot B}{2P} = B \quad \text{vs} \quad \text{ridge} = \frac{989 \text{ TFLOPS}}{3.35 \text{ TB/s}} \approx 300$$

Batch 1 gives ~1, far below 300, so decode is memory-bound and batching is nearly free speed. Prefill puts ~500 prompt tokens through one read, above the ridge, so it's compute-bound:

$$t_{\text{prefill}} \approx \frac{N_{\text{prompt}} \cdot 2P}{\text{peak FLOPS}} = \frac{500 \times 16 \text{ GFLOP}}{989 \text{ TFLOPS}} \approx 8 \text{ ms}$$

3. Cost = what you pay per hour ÷ what you make per hour

$$\frac{\$}{1\text{M tokens}} = \frac{\$ \text{ per GPU hour}}{\text{tok/s} \times 3600} \times 10^6 \quad \frac{3}{200 \times 3600} \times 10^6 \approx 4.17 \quad \frac{3}{3000 \times 3600} \times 10^6 \approx 0.28$$

4. Why batching isn't free: every sequence brings its own KV cache ♦

Each layer stores one K and one V vector per KV head for every past token:

$$\text{KV/token} = 2 \times L \times H_{\text{kv}} \times d_{\text{head}} \times \text{bytes} = 2 \times 32 \times 8 \times 128 \times 2 = 128 \text{ KB}$$

A step reads the weights once plus every sequence's cache (context length C), so steps slow down as the batch grows:

$$t_{\text{step}}(B) \approx \frac{2P + B \cdot C \cdot \text{KV/token}}{\text{BW}}$$

Prefill vs decode

	Prefill	Decode
Work	Whole prompt in one parallel pass	One token per sequence per step, serial
Bottleneck	Compute (FLOPs)	Memory bandwidth (bytes)
User sees it as	TTFT	Gap between tokens (ITL / TPOT)
Grows with	Prompt length	Output length, and batch × context (KV reads)

Symptom → lever (for "how would you fix it?" questions)

Symptom	Likely cause	First levers
High \$/token, GPU underused	Batch too small	Scheduling: batch more requests per step
Batch capped by out-of-memory	KV cache fills memory	Memory: pack KV tightly, reuse shared prefixes
Slow tokens even at low load	Too many weight bytes per step	Precision: 8/4-bit weights
High TTFT	Long prompts or queueing	Shorter prompts, more replicas (routing fixes: see 02)
Model doesn't fit one GPU	Size	Parallelism across GPUs, or precision

Rapid-fire Q&A

Question	Crisp answer
Why does batching help, and what does it cost?	One weight read serves all B sequences. The cost is KV memory and slower steps (derivation 4).
♦ Max tok/s for 70B BF16 on 8×H100?	Derivation 1: $140 \text{ GB} \div (8 \times 3.35 \text{ TB/s}) \approx 5.2 \text{ ms/step}$, so $\leq \sim 190 \text{ tok/s}$ before communication overhead.
Cost too high: where do you start?	Measure (utilization, batch size, TTFT/ITL) before changing anything; then check off-GPU wins such as a smaller model, shorter prompts, caching.

TRAP

- "Just call `generate()`" or "buy a faster GPU": both ignore the scheduler, memory and setup.
- Trading away quality: a faster but worse answer is not a win.

02 · The Inference Stack: Model to GPU to Production

Full notes

Say it in 30 seconds

A request crosses about eight layers. The model defines the math; kernels, CUDA and the GPU set how fast it runs, bounded by bytes ÷ bandwidth. The inference engine batches requests every step and manages KV cache memory. The API server, router and autoscaler pick the replica and the replica count. Nearly all time is GPU time; the upper layers decide how well it's used.

Only numbers worth memorizing

- **H100**: 80 GB HBM, 132 SMs. **Llama 3.1 8B**: 32 layers. Routing ~1 ms; decode ~5 ms/token.

Derive, don't memorize

Every derivation here is the same idea: **time = bytes ÷ bandwidth**, or **capacity = bytes ÷ bytes-per-item**. Per-step time and KV per token: see 01, derivations 1 and 4.

1. KV capacity = memory left after the weights ÷ cache per token ♦

$$N_{\text{tokens}} \approx \frac{\text{HBM} - \text{weights}}{\text{KV/token}} = \frac{80 - 16 \text{ GB}}{128 \text{ KB}} \approx 488\text{k} \approx 119 \text{ requests} \times 4\text{k}$$

INTUITION

That token budget, not compute, caps the batch. It's why engines manage KV in blocks and why quantizing weights or KV frees room for more users.

2. Launch overhead = kernels × launch cost, against the step time ♦

$$\frac{N_{\text{kernels}} \times t_{\text{launch}}}{t_{\text{step}}} = \frac{300 \times 4 \mu\text{s}}{4.8 \text{ ms}} \approx 25\%$$

INTUITION

Illustrative numbers, but the shape is real: at small batch, CPU launch time is a big slice of each step. Fusion (and ♦ CUDA graphs) cut it.

3. Cold start = weight bytes ÷ load bandwidth ♦

$$t_{\text{cold}} \approx \frac{16 \text{ GB}}{2 \text{ GB/s}} = 8 \text{ s}$$

INTUITION

Assumes ~2 GB/s from local disk; from network storage it's often slower. Scaling reacts late, so autoscalers must scale before the queue explodes.

Layer → question → what to watch

Layer	Answers	Tools	Watch / lever
Model + framework	What to compute	Weights, recipe, tokenizer, PyTorch	Model size, precision
Kernels + CUDA + GPU	How fast the math runs	cuBLAS, FlashAttention, CUDA	Bytes moved, fusion, launches
Inference engine	Who runs each step; where memory goes	vLLM, SGLang, TensorRT-LLM	Batch size, KV usage
API server	Speaks HTTP, tokenizes, streams	OpenAI-compatible server	Streaming latency
Router + autoscaler	Which replica; how many	Prefix-aware router	Queue length, cache hits

Symptom → layer to investigate

Symptom	Layer	First fix
GPU idle between kernels, CPU busy	Kernels	Fuse kernels; cut launches
Same math slower than expected	Kernels	Better kernels (cuBLAS, FlashAttention)
Repeated system prompts inflate TTFT	Router	Prefix-aware routing
Queue grows while CPU looks idle	Autoscaler	Scale on queue length and KV use
New replicas arrive too late	Fleet	Derivation 3: scale earlier, load faster

Rapid-fire Q&A

Question	Crisp answer
What does an inference engine add over PyTorch?	A per-step scheduler (batching) and a KV cache memory manager.
Why not autoscale on CPU?	The work is on the GPU; the CPU idles. Watch queue length and KV use.
Why is a round-robin load balancer wasteful?	It scatters shared prompts, so each replica re-prefills them.
What's a replica?	One engine on its GPU(s), behind an API server.
Where does a request's time go?	Mostly GPU (prefill, then ~5 ms per decode step); upper layers decide utilization.

TRAP

- Treating LLM serving like a stateless web app: round-robin routing and CPU-based autoscaling.
- Assuming the model file determines speed: it only defines the math.